

EXAM-READY NOTES — every topic + a proper example

Definition first, then an **example on its own line**. Many examples are your lecturer's own (Therac-25, autopilot, divide-by-zero, auto-save) — use those; he set them. Cover this and you can answer any question with the example he wants to see.

CHAPTER 1 — Basics of Software Quality Assurance

Software — not just a program; it is programs + procedures + documentation + data. Two types: **Generic (Buy → COTS)** and **Customized (Build → bespoke)**. *Example:* MS Office bought off the shelf = generic/COTS; a bank's custom loan-approval system = customized.

Software quality — ISO/IEC 9126: totality of functionality and features that satisfy stated or implied needs. IEEE 610: degree to which it meets requirements and user expectations. *Example:* A calculator app that always returns correct results and is easy to use = high quality.

Three project challenges — Time, Cost, Scope. *Example:* An app delivered late (Time), over budget (Cost), and still buggy (Scope) fails all three.

Famous defects — *Example: Therac-25* (1986, AECL): a software bug caused radiation-overdose deaths. *Ariane 5* (4 June 1996): the rocket exploded 37 seconds after launch from a control-software fault — the most expensive computer bug in history.

Software testing — executing a system under specified conditions to find defects and verify requirements.

Example: Entering a wrong password to confirm the login screen rejects it.

Static analysis (manual) — examine code/documents without running them; reason over all possible behaviours (code review, inspection, algorithm analysis). Slow. *Example:* Reading a function line-by-line in a review and spotting a missing null check.

Dynamic analysis (automated) — actually execute the program to expose failures. *Example:* Running the app and watching it crash on a bad input. *(Static + dynamic are complementary.)*

SQA (Software Quality Assurance) — planned, process-oriented activity that **assures** standards/processes are right; an umbrella activity; does **not** run the program; aims to **prevent** defects. *Example:* Deciding that every module must be code-reviewed before it is merged.

SQC (Software Quality Control) — product-oriented activity that **checks** the product by executing it; aims to **find and resolve** defects; only the testing team. *Example:* Actually running the login test cases and logging the bugs found.

Verification vs Validation — Verification = "doing the things right" (process; SQA). Validation = "results are what you expected" (product; SQC). *Example:* Verification = checking the code follows the design; Validation = checking the finished feature satisfies the user.

Difficulties in achieving quality — Size, Complexity, Environmental stress, Flexibility, Cost. *Example:* A logic error in autopilot "set landing mode" (complexity); a results website tested at maximum load (environmental stress).

Software Quality Engineering (SqE) — ensure quality through V&V. Three stages: **Planning → Execution → Decision**. Alternatives to testing: **defect prevention, formal verification (inspection/review/walkthrough), fault tolerance**. *Example:* Plan the tests, run them, then decide pass/fail; auto-save is a fault-tolerance alternative.

Error → Fault → Failure → Defect — Error = human action producing a wrong result → Fault = the wrong step/data in the code → Failure = the system can't perform its function. All three collectively = Defects ("bugs"). *Example:* A programmer forgets to handle divide-by-zero (**error**) → the faulty line stays in the code (**fault**) → the app crashes when a user divides by 0 (**failure**).

Complete/Exhaustive testing — testing every input/path; nearly impossible (input domain too large, design too complex, can't build every environment). *Example:* A login field accepts endless username/password combinations — you can't test them all.

CHAPTER 2 — Software Quality

Quality perspectives — External/Consumer sees a **black box** (behaviour only); Internal/Producer sees a **white/clear box** (the workings). *Example:* A user just opens the app and uses it (black box); the developer sees the source code (white box).

Five views of quality (Kitchenham & Pfleeger) — 1. **Mystical** — quality is recognized by experience but can't be defined. *Example:* "I just know good software when I use it." 2. **User** — meets user needs; good if it satisfies many users. *Example:* An app with high ratings from most users. 3. **Manufacturing** — conformance to requirements, "right the first time" (basis of **CMM & ISO 9001**). *Example:* A factory-style process that builds to spec with no rework. 4. **Product** — good internal properties give good external ones. *Example:* Clean modular code → fewer crashes in the field. 5. **Value-based** — willingness to pay; merges excellence (quality) and worth (value). *Example:* A "good enough for the price" phone.

Why measure quality — Baseline, quality improvement based on cost, know present level for planning. *Example:* Baseline goal = "a user can extract the needed info from the website within 20 minutes."

Quality Factor — an external behavioural characteristic of the system (from McCall's model). *Example:* Correctness, Reliability, Efficiency, Performance.

Quality Criterion — an internal attribute of a factor, tied to development. *Example:* Modularity, Testability, Maintainability, Reusability.

Key factors (with slide examples) — - **Integrity/Security** — control unauthorized access. *Example:* Only users with Auditor privilege can view customer transaction histories. - **Efficiency** — resources/code needed for a function. *Example:* At least 25% of processor and RAM stays unused at peak load. - **Usability** — effort to learn/operate. *Example:* A trained user submits a request in an average of 4 (max 6) minutes. - **Reliability** — probability of running **without failure for a specific period of time**. *Example:* No more than 5 of 1000 runs may be lost to software failures.

Key criteria (with slide examples) — - **Maintainability** — effort to locate and fix a defect. *Example:* A programmer can update a report to new regulations in ≤20 hours. - **Testability** — effort to ensure it works. *Example:* A module's cyclomatic complexity must not exceed 20. - **Portability** — effort to move to another environment. *Example:* Moving an app from Windows to Linux. - **Reusability** — parts reusable elsewhere. *Example:* Chemical-structure input functions reusable in other applications. - **Interoperability** — effort to couple with another system. *Example:* Importing a structure from the ChemiDraw tool.

ISO-9126 — the most influential framework; hierarchical (characteristics + non-overlapping sub-characteristics).

Six top-level characteristics: *Example:* **Functionality** (what is needed) · **Reliability** (functions correctly) ·

Usability (effort to use) · **Efficiency** (resources needed) · **Maintainability** (correct/improve/adapt) · **Portability** (environment to environment). **Security is NOT one of them.**

Quality expectations — Consumer = "good enough" for the **price** (fit-for-use + conformance); Producer = "good enough" for the **cost**. *Example:* A user wants a phone that's worth its price; the maker wants it profitable to build.

CHAPTER 3 — Maturity Models

Software process — a set of activities (methods, procedures, practices) executed to develop products; benefits: repeatable, measurable, improvable. *Example:* Following the same requirement→design→code→test steps on every project so results are consistent.

Why improve a process — Quality, Lead Time, Cost. *Example:* A better test process finds more defects (quality), faster (lead time), for less money (cost).

Three models — **CMM** evaluates the **development** process; **TMM** evaluates the **testing** process; **TPI** improves the **testing** process.

CMM — 5 levels (how mature the development process is): 1. **Initial** — chaotic; depends on individuals; not repeatable. *Example:* A startup with no defined process; success rides on one star coder. 2. **Repeatable** — basic project management; successes repeatable. *Example:* The team reuses the same planning/tracking each project. 3. **Defined** — its own documented standard process. *Example:* A company-wide SDLC handbook everyone follows. 4. **Managed** — monitors and controls processes via **data collection and analysis**. *Example:* Tracking defect-density metrics to control quality. 5. **Optimizing** — continuous improvement via feedback and innovation. *Example:* Using past-project data to prevent defects and adopt new methods.

TPI — improve a test process (4 steps): determine area → evaluate current state (baseline) → identify next desired state → implement changes. *Example:* Find that regression takes too long (area) → measure current time (baseline) → target automation (next state) → introduce the tool (implement).

TMM (pioneered by Ilene Burnstein) — 5 levels; each level has Maturity goals, Supporting goals, and **ATRs** (Activities, Tasks, Responsibilities): 1. **Initial** — testing after code, ad hoc, not tracked. *Example:* Testing only to "show it works," no plan. 2. **Phase Definition** — testing/debugging goals; initiate **test planning**. *Example:* Writing a test plan with objectives and risks. 3. **Integration** — establish a **test group**; integrate testing into the lifecycle. *Example:* Hiring a dedicated QA team and testing at every stage. 4. **Management and Measurement** — org-wide review; test management; evaluate quality. *Example:* Reviewing and measuring quality across all projects. 5. **Optimization/Defect Prevention & QC** — defect prevention, statistical quality control, process optimization. *Example:* Using data to stop defects before they happen.

CHAPTER 4 — Software Quality Assurance

Three generic QA categories: - **Defect Prevention** — stop faults being **injected**. *Example:* Default values and dropdown options so a user can't enter bad input. - **Defect Reduction** — detect and **remove** injected faults. *Example:* Code inspection and testing. - **Defect Containment** — contain failures / limit damage. *Example:* Auto-save and exception handling so a crash doesn't lose the user's work.

Defect prevention — 2 ways: - **Error source removal** — remove root causes via **education and training** (people-based). *Example:* Training developers to avoid a common misconception. - **Error blocking** — directly block wrong actions. *Example:* The program won't let a user set the divisor to 0; Excel data validation.

Defect reduction — techniques: Inspection (most common **static** technique — informal walkthroughs vs formal inspections), Testing, Boundary value analysis, Control/data-flow analysis, Simulation/Prototyping. *Example:* A formal inspection by several reviewers finds design flaws before coding; simulation tests an autopilot safely.

Defect containment — 2 ways: - **Software fault-tolerance** — Recovery (**rollback and redo**) and **NVP (N-Version Programming)**. *Example:* Rolling a database back to its last good state after an error. - **Safety assurance & failure containment.**

Safety terms — **Safety** = accident-free; **Accident** = failure with severe consequences; **Hazard** = pre-condition to an accident; Safety assurance = hazard analysis + elimination/reduction/control + damage control. *Example:* Autopilot safety eliminates pilot error; a hazard (icy runway condition) is handled before it becomes an accident.

Process variations — Waterfall, Iterative (QA in increments), Spiral (QA + risk management), Agile (XP — TDD, pair programming), Maintenance (defect handling). *Example:* Agile/XP writes tests first (TDD) and pairs two developers at one screen.

DC → V&V mapping — Defect prevention ≈ both; Defect reduction ≈ mostly verification (**unit = verification, beta = validation**); Defect containment ≈ mostly validation. *Example:* Unit testing checks internal correctness (verification); beta testing checks user satisfaction (validation).

CHAPTER 5 — Software Quality Engineering (SQE)

SQE — meet/exceed quality expectations through appropriate QA activities while **minimizing cost and project risk**; an integral part of software engineering. *Example:* Choosing which tests to run to hit the reliability goal without blowing the budget.

Generic testing process (3 phases) — **Pre-QA** (planning, most key decisions), **In-QA** (test execution, handle defects), **Post-QA** (measure, assess, improve — runs **parallel** to QA, not after). *Example:* Plan tests → run them and log bugs → analyze results to improve next time.

Test Plan — a **high-level** document: objectives, scope, approach, resources, schedule of testing. *Example:* "We will test the login module for 2 weeks using 3 testers, covering functionality and load."

Test Case — a **low-level** document: one input/action and its expected result. *Example:* Test Case FR_10 — "Verify login with valid username and password → user logs in successfully (Pass)."

Test Suite — the **macro-level** collection of test cases run until stopping criteria are met. *Example:* All 40 login test cases run together as one suite.

Regression testing — reuse test cases from earlier versions (design **no** new ones) to check a change broke nothing. *Example:* After adding a "Remember me" checkbox, re-running the old login tests to confirm login still works.

Testing team models — - **Vertical** — organized around **one product**; dedicated people do many testing tasks for it. *Example:* A team that does all testing for the banking app. - **Horizontal** — **one kind of testing** across many products. *Example:* A security-testing team that tests every product in the company. - **Mixed** — both; used by large organizations.

CHAPTER 6 — Software Testing

Testing — execute the software and observe behaviour; on failure, locate and fix the fault; else gain confidence.

Example: Running a payment and checking the balance updates correctly.

Seven principles of testing (high-yield — give an example for each):

1. **Presence not absence** — testing shows defects exist, never proves none remain. **Example:** Passing all tests doesn't guarantee the app is bug-free.
2. **Exhaustive testing is impossible** — use risk/priority instead. **Example:** You can't test every possible input, so test the most important cases.
3. **Early testing** — earlier defects are cheaper to fix. **Example:** A requirement error caught in design costs far less than after release.
4. **Defect clustering** — a few modules hold most defects. **Example:** 80% of bugs come from the payment and login modules.
5. **Pesticide paradox** — repeating the same tests stops finding new bugs; revise them. **Example:** The same regression suite finds nothing new, so you add fresh tests.
6. **Testing is context dependent** — different software, different testing. **Example:** Autopilot software is tested far more rigorously than an e-commerce site.
7. **Absence-of-errors fallacy** — a bug-free build is useless if it doesn't meet user needs. **Example:** A flawless app nobody wants to use is still a failure.

Four testing levels + who does each:

- **Unit — Developer:** test individual units in isolation. **Example:** Testing one `calculateTax()` method.
- **Integration — Tester:** combine modules and test. **Example:** Checking the cart module talks correctly to the payment module.
- **System — Tester:** test the whole system. **Example:** Testing the full app's functionality and load together.
- **Acceptance — End-user:** verify customer expectations. **Example:** The client confirms the app does what they ordered.

Regression testing — selects/prioritizes/executes existing tests; designs none. **Example:** Re-running old tests after a bug fix to ensure nothing else broke.

Developer vs Tester roles — Developer builds (constructive, driven by delivery); Tester tries to break it (destructive, driven by quality); debugging is a developer activity. **Example:** The developer's win is shipping the feature; the tester's win is finding a bug in it.

Testing techniques:

- **White-box** — structural/implementation-based; examines source code (**control flow + data flow**); done by **developers**; early/small units. **Example:** Testing every branch of an `if/else` in a function.
- **Black-box** — functional/specification-based; tests the **external interface** (input → output); done by **professional/third-party testers**; late/large systems. **Example:** Typing a username/password and checking the result without seeing the code.
- **Gray-box** — mixed (middle levels). **Example:** Testing procedures individually as black-box, but their interconnection as white-box at module level.

White-box vs Black-box (dimensions): basis (code vs spec), object (small vs large), timeline (early vs late), tester (developer vs professional). WBT defects are easier to fix; BBT catches omission/design problems WBT misses.

When to stop testing (exit criteria) — "no more defects found" is **NOT** valid. Use **resource-based** (out of time/money — irresponsible) or **quality-based** (quality goals reached) or activity completion. **Example:** Stop when reliability and usability goals are met, not just when the clock runs out.

Manual vs Automated testing:

- **Manual** — human-run; rigorous but hard to repeat, costly, slow. **Example:** A tester manually clicking through a new screen.
- **Automated** — tool-run; fast, repeatable, reliable, reusable. **Example:** A script re-running 500 regression tests overnight.

What to automate — every-build/regression, data-driven, GUI-internal, stress/load tests. **What NOT to automate** — usability, logical errors, spec/design docs, one-time/ASAP, ad-hoc, tests without predictable results. **Automation cannot replace manual testing.** **Example:** You can automate a nightly regression run, but "how easy is this app to use?" (usability) needs a human.